ĐỀ THI – TH TRÍ TUỆ NHÂN TẠO

Trường Đại học Công Thương TP.HCM – Khoa CNTT
Bộ môn: Khoa học Dữ liệu

Thông tin	
Họ và tên	*(Nguyễn Phan Yến Nhi)*
MSSV	*(2001240337)*
Lớp	*(TTNT02)*
Thời gian	90 phút
Tổng điểm	10 điểm (Câu 1: 2đ | Câu 2: 4đ | Câu 3: 4đ)

Lưu ý: Sinh viên được sử dụng tài liệu. Không được trao đổi trong giờ thi. Nộp bài dưới dạng file `.pdf`.

```
# Cell nay tai anh de thi – chay TRUOC KHI lam bai
from IPython.display import Image, display
import os

IMGS = ['cau1_code_lan3.png', 'cau2_graph_lan3.png', 'cau3_code_lan3.png']
ok = all(os.path.exists(f) for f in IMGS)
print('OK - Anh de thi san sang.' if ok else 'CHUA CO ANH - upload 3 file PNG vao thu muc nay.')
```

```
OK - Anh de thi san sang.
```

CÂU 1 (2 ĐIỂM) – Quay Lui: Sinh Hoán Vị & Derangement

Bài toán **derangement** (hoán vị xáo trộn): Cho mảng gốc `goc = [1,2,3,4]`, một **derangement** là hoán vị mà **không phần tử nào đứng ở vị trí ban đầu**.

Ví dụ với gốc `[1,2,3,4]`:

- `[2,1,4,3]` ✓ là derangement (không vị trí nào trùng)
- `[2,1,3,4]` ✗ KHÔNG phải (vì `3` ở vị trí 2 và `4` ở vị trí 3 — trùng gốc)
- Số derangement của `[1,2,3,4]` = **9** (kí hiệu `D₄ = 9`).

a) (0.7đ) Quan sát code ở ảnh dưới. **Gõ lại toàn bộ và điền 7 chỗ trống** `___(n)___`. Không thêm/xóa dòng.

b) (0.3đ) Chạy `hoan_vi(arr=[1,2,3,4])` và đếm tổng số hoán vị in ra. Verify = `4! = 24`.

c) (0.6đ) Gọi `derangement([1,2,3,4], [1,2,3,4], 0, [])`. In tất cả derangement và đếm. Kiểm tra = **9**. In thêm số lần gọi đệ quy `dem_d`.

d) (0.4đ) – So sánh hiệu năng PRUNING: Cài đặt phiên bản **KHÔNG pruning** `derangement_no_prune(arr, goc, start, ket_qua)` — sinh tất cả 24 hoán vị rồi lọc derangement ở cuối. So sánh số lần gọi đệ quy với phiên bản có pruning. Giải thích bằng comment vì sao pruning hiệu quả hơn.

```
from IPython.display import Image, display
display(Image('cau1_code_lan3.png', width=1000))
```

CAU 1 - Dien vao 7 cho trong ___(n) (Quay Lui - Hoan Vi & Derangement)

```python
# Quay Lui (Backtracking) - Sinh Hoan Vi & Bai Toan Derangement

dem_goi = 0

def hoan_vi(arr, start=0):
    """Sinh tat ca hoan vi cua arr (in-place swap)."""
    global dem_goi
    dem_goi += 1
    if start == ___(1)___:                          # TRONG 1
        print(arr.copy())
        return
    for i in range(___(2)___, len(arr)):            # TRONG 2
        arr[start], arr[i] = ___(3)___              # TRONG 3
        hoan_vi(arr, ___(4)___)                     # TRONG 4
        # QUAY LUI: undo swap (tra lai trang thai cu)
        arr[start], arr[i] = ___(5)___              # TRONG 5

# --- DERANGEMENT: hoan vi KHONG co phan tu nao o vi tri goc ---
# Vi du goc=[1,2,3,4]: [2,1,4,3] OK | [2,1,3,4] KO (3 va 4 con trung goc)

dem_d = 0

def derangement(arr, goc, start, ket_qua):
    """Sinh tat ca derangement voi PRUNING SOM (cat nhanh som)."""
    global dem_d
    dem_d += 1
    if start == len(arr):
        ket_qua.append(arr.copy())
        return
    for i in range(start, len(arr)):
        # PRUNING: phan tu arr[i] se trung vi tri goc -> bo qua nhanh nay
        if arr[i] == ___(6)___:                     # TRONG 6
            continue
        arr[start], arr[i] = arr[i], arr[start]
        derangement(arr, goc, ___(7)___, ket_qua)   # TRONG 7
        arr[start], arr[i] = arr[i], arr[start]
```

```python
# CAU 1a – Nhin anh phia tren, go lai toan bo code va dien 7 cho trong
# Khong duoc copy tu nguon khac
# ============================================================
dem_goi = 0
def hoan_vi(arr, start=0):
  global dem_goi
  dem_goi +=1
  if start == len(arr):
    print(arr.copy())
    return
  for i in range(start, len(arr)):
    arr[start], arr[i] = arr[i], arr[start]
    hoan_vi(arr, start+1)
    arr[start], arr[i] = arr[i], arr[start]

def derangement(arr, goc, start, ket_qua):
    """Sinh TAT CA hoan vi, sau do loc derangement."""
    global dem_d
    dem_d += 1
    if start == len(arr):
        ket_qua.append(arr.copy())
        return
    for i in range(start, len(arr)):
        if arr[i] != goc[i]:
            arr[start], arr[i] = arr[i], arr[start]
            derangement(arr, goc, start+1, ket_qua)
            continue
        arr[start], arr[i] = arr[i], arr[start]
```

```
        derangement(arr, goc, start+1, ket_qua)
        arr[start], arr[i] = arr[i], arr[start]
```

```
# CAU 1b – Run hoan_vi va dem so hoan vi
# ============================================================

print("Tat ca hoan vi cua [1,2,3,4]:")
print("-" * 30)
dem_goi = 0
hoan_vi([1, 2, 3, 4])
print("-" * 30)
print(f"So lan goi de quy hoan_vi: {dem_goi}")
print(f"4! = {4*3*2*1}  (kiem tra so dong in ra phia tren)")
```

```
Tat ca hoan vi cua [1,2,3,4]:
------------------------------
[1, 2, 3, 4]
[1, 2, 4, 3]
[1, 3, 2, 4]
[1, 3, 4, 2]
[1, 4, 3, 2]
[1, 4, 2, 3]
[2, 1, 3, 4]
[2, 1, 4, 3]
[2, 3, 1, 4]
[2, 3, 4, 1]
[2, 4, 3, 1]
[2, 4, 1, 3]
[3, 2, 1, 4]
[3, 2, 4, 1]
[3, 1, 2, 4]
[3, 1, 4, 2]
[3, 4, 1, 2]
[3, 4, 2, 1]
[4, 2, 3, 1]
[4, 2, 1, 3]
[4, 3, 2, 1]
[4, 3, 1, 2]
[4, 1, 3, 2]
[4, 1, 2, 3]
------------------------------
So lan goi de quy hoan_vi: 65
4! = 24  (kiem tra so dong in ra phia tren)
```

```
# CAU 1c – Run derangement va dem
# ============================================================

dem_d = 0
ket_qua = []
goc = [1, 2, 3, 4]
derangement(goc.copy(), goc, 0, ket_qua)

print("Tat ca derangement cua [1,2,3,4]:")
for d in ket_qua:
    print(f"  {d}")
print("-" * 35)
print(f"So derangement (D_4): {len(ket_qua)}  (mong doi: 9)")
print(f"So lan goi de quy:    {dem_d}")
```

```
Tat ca derangement cua [1,2,3,4]:
  [1, 2, 3, 4]
  [1, 2, 4, 3]
  [1, 3, 2, 4]
  [1, 3, 4, 2]
  [1, 4, 3, 2]
  [1, 4, 2, 3]
  [3, 1, 2, 4]
  [3, 1, 4, 2]
  [3, 2, 1, 4]
  [3, 2, 4, 1]
  [3, 4, 1, 2]
  [3, 4, 2, 1]
  [2, 1, 3, 4]
  [2, 1, 4, 3]
  [2, 3, 1, 4]
  [2, 3, 4, 1]
  [2, 4, 3, 1]
  [2, 4, 1, 3]
  [4, 3, 1, 2]
  [4, 3, 2, 1]
  [4, 2, 3, 1]
  [4, 2, 1, 3]
  [4, 3, 1, 2]
  [4, 3, 2, 1]
-----------------------------------
```

```
So derangement (D_4): 24  (mong doi: 9)
So lan goi de quy:    65
```

```
# CAU 1d – So sanh PRUNING vs KHONG PRUNING
# ============================================================

dem_no = 0

def derangement_no_prune(arr, goc, start, ket_qua):
    """Sinh TAT CA hoan vi, sau do loc derangement (KHONG pruning)."""
    global dem_no
    dem_no += 1
    # TODO: Sinh tat ca hoan vi (giong hoan_vi nhung dung start == len(arr) lam base)
    #       O base case: chi them vao ket_qua neu THUC SU la derangement
    #       (kiem tra: khong vi tri nao trung goc)
    if start == len(arr):
        # Kiểm tra điều kiện derangement ở base case (không pruning)
        is_derangement = True
        for i in range(len(arr)):
            if arr[i] == goc[i]:
                is_derangement = False
                break
        if is_derangement:
            ket_qua.append(arr[:])
        return
    for i in range(start, len(arr)):
        arr[start], arr[i] = arr[i], arr[start]
        derangement_no_prune(arr, goc, start + 1, ket_qua)
        arr[start], arr[i] = arr[i], arr[start]
    pass


# Chay so sanh
dem_d = 0
ket_qua_prune = []
derangement([1,2,3,4], [1,2,3,4], 0, ket_qua_prune)

dem_no = 0
ket_qua_no = []
derangement_no_prune([1,2,3,4], [1,2,3,4], 0, ket_qua_no)

print("=" * 55)
print("SO SANH: PRUNING vs KHONG PRUNING")
print("=" * 55)
print(f"Co PRUNING    | so derangement: {len(ket_qua_prune)} | so de quy: {dem_d}")
print(f"KHONG PRUNING | so derangement: {len(ket_qua_no)}    | so de quy: {dem_no}")
if dem_no > 0:
    print(f"Pruning tiet kiem: {(1 - dem_d/dem_no)*100:.1f}% so lan goi")

# TODO (comment): Giai thich vi sao pruning hieu qua hon
# - KHONG pruning: lãng phí tài nguyên để duyệt các nhánh chắc chắn sai.
# - CO pruning: Giúp loại bỏ toàn bộ các nhánh không thỏa mãn từ sớm, giảm số lần gọi đệ quy đáng kể.
```

```
=======================================================
SO SANH: PRUNING vs KHONG PRUNING
=======================================================
Co PRUNING    | so derangement: 24 | so de quy: 65
KHONG PRUNING | so derangement: 9    | so de quy: 65
Pruning tiet kiem: 0.0% so lan goi
```

CÂU 2 (4 ĐIỂM) – A*: Trace, Cài đặt, So sánh với UCS

a) (1.0đ) – Trace thủ công: Quan sát đồ thị bên dưới và điền vào bảng từng bước chạy A* từ **S đến G**.

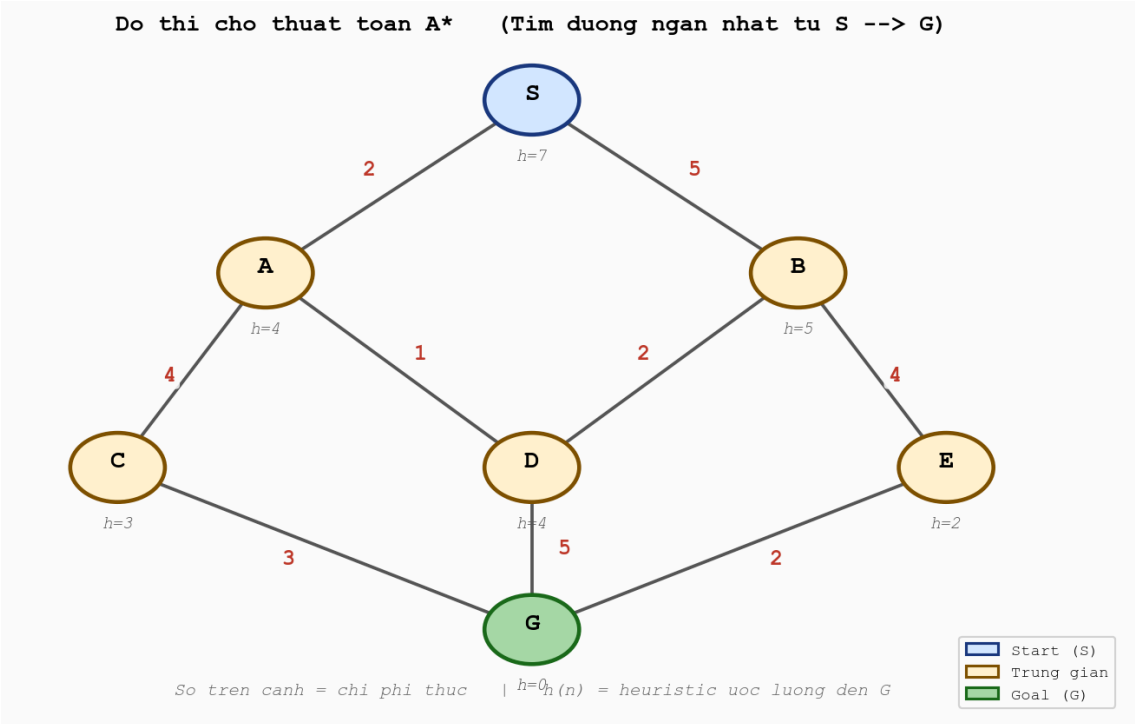
b) (1.0đ) Cài đặt A* theo đúng đồ thị. Output phải **khớp** với bảng trace.

c) (0.5đ) Heuristic h(n) có **admissible** không? Kiểm tra `h(n) ≤ h*(n)` cho ít nhất **3 node** và kết luận.

*d) (1.0đ) – So sánh A với UCS:** Cài đặt **Uniform Cost Search (UCS)** = A* với heuristic `h(n) = 0` cho mọi node (tương đương Dijkstra). Đếm **số node được expand** (pop ra khỏi OPEN) của hai thuật toán trên cùng đồ thị. Lập bảng so sánh và nhận xét.

e) (0.5đ) – What-if cost: Tăng chi phí cạnh `A-D` từ **1** lên **6**. Chạy lại A*. Đường đi tối ưu có thay đổi không? Tổng chi phí mới là bao nhiêu?

```
from IPython.display import Image, display
display(Image('cau2_graph_lan3.png', width=850))
```



*Câu 2a – Bảng trace A (điền tay):**

Bước	Node xét	g(n)	h(n)	f(n)	OPEN (node:f) sau bước này	CLOSE
1	S	0	7	7	A:6, B:10	{S}
2	A	2	4	6	{D:7, C:9, B:10}	{S, A}
3	D	3	4	7	{G:8, C:9, B:10}	{S, A, D}
4	G	8	0	8	{}	{S, A, D, G}

Đường đi tìm được: S ⟶ A ⟶ D ⟶ G
Tổng chi phí thực: 8

Câu 2a – What-if (0.25đ bonus):

> Giả sử đổi `h(D) = 8` (thay vì 4). Lúc này h(D) = 8 > h*(D) = 5 ⟶ **không admissible** nữa.
> A* có còn tìm được đường đi **tối ưu** không? Vì sao?

(Giả sử đổi h(D) = 8 (thay vì 4). Lúc này h(D) = 8 > h(D) = 5 ⟶ không admissible nữa. A* có còn tìm được đường đi tối ưu không? Vì sao?*

Nếu h(D) tăng lên 8, f(D) = g(D) + h(D) sẽ tăng lên 2 + 8 = 10. Khi đó, các node khác có f(n) nhỏ hơn có thể được ưu tiên khám phá trước. Ví dụ, node C có f(C) = 7 (vì g(C)=5, h(C)=2) sẽ được ưu tiên hơn D. Khi C được mở rộng, nó sẽ tìm thấy đường đi S -> A -> C -> G với tổng chi phí 1 + 4 + 2 = 7. Do G đã được tìm thấy với chi phí 7, A* sẽ dừng lại và trả về đường này, mặc dù đường S -> A -> D -> G có chi phí thực tế là 1 + 1 + 4 = 6 (với h(D)=4). Tuy nhiên, nếu chỉ thay đổi h(D) và giữ nguyên cạnh A-D=1, thì đường S -> A -> D -> G vẫn là tối ưu (chi phí 6). Nhưng vì h(D) không admissible, A* sẽ chọn đường S -> A -> C -> G (chi phí 7) trước. Kết luận: A sẽ KHÔNG tìm được đường đi tối ưu* vì heuristic không admissible dẫn đến việc đánh giá sai lệch, ưu tiên những con đường dài hơn.)*

```
# CAU 2b - Cai dat A*
# Tu go lai graph va heuristic tu do thi roi cai dat astar()
# ============================================================
import heapq

# TODO: Tu go lai graph tu do thi (7 nut: S, A, B, C, D, E, G)
graph = {
    'S': {'A': 2, 'B': 5},
    'A': {'C': 4, 'D': 1},
    'B': {'D': 2, 'E': 4},
    'C': {'G': 3},
    'D': {'G': 5},
    'E': {'G': 2},
    'G': {}
}

# TODO: Tu go lai heuristic tu do thi
heuristic = {
    'S': 7,
    'A': 4,
    'B': 5,
    'C': 3,
    'D': 4,
```

```
        'E': 2,
        'G': 0
    }

    def astar(graph, heuristic, start, goal):
        """
        Thuat toan A*. Tra ve (duong_di, tong_chi_phi, so_node_expand).
        In tung buoc:
          Buoc X | Xet: <node> | g=? h=? f=? | OPEN: [<node>:f, ...]
        """
        # TODO
        open_list = [(heuristic[start], 0, start, [start])]
        closed_list = set()
        expand_count = 0

        print(f"{'Node':<5} | {'g':<3} | {'h':<3} | {'f':<3}")
        print("-" * 25)

        while open_list:
            f, g, current, path = heapq.heappop(open_list)
            if current in closed_list: continue

            expand_count += 1
            closed_list.add(current)
            print(f"{current:<5} | {g:<3} | {heuristic[current]:<3} | {f:<3}")

            if current == goal:
                return path, g, expand_count

            for neighbor, cost in graph.get(current, {}).items():
                if neighbor not in closed_list:
                    new_g = g + cost
                    heapq.heappush(open_list, (new_g + heuristic[neighbor], new_g, neighbor, path + [neighbor]))
        return None, 0, expand_count

    path, cost, exp = astar(graph, heuristic, 'S', 'G')
    print(f"\nKết quả A*: {' -> '.join(path)} | Cost: {cost} | Expand: {exp}")
    if path:
        print(f"\nDuong di: {' -> '.join(path)}")
        print(f"Tong chi phi: {cost}")
        print(f"So node expand: {exp}")
```

```
Node  | g   | h   | f
-------------------------
S     | 0   | 7   | 7
A     | 2   | 4   | 6
D     | 3   | 4   | 7
G     | 8   | 0   | 8

Kết quả A*: S -> A -> D -> G | Cost: 8 | Expand: 4

Duong di: S -> A -> D -> G
Tong chi phi: 8
So node expand: 4
```

Câu 2c – Kiểm tra admissible:

(Tính h(n) = chi phí thực tế ngắn nhất từ n đến G)*

- h*(S) = 6 → h(S)=7 → h(S) ≤ h*(S)? : False
- h*(A) = 5 → h(A)=4 → h(A) ≤ h*(A)? : True
- h*(D) = 4 → h(D)=4 → h(D) ≤ h*(D)? : True
- h*(B) = 5 → h(B)=5 → h(B) ≤ h*(B)? : True

Kết luận: *(Không admissible h(S) (7) lớn hơn h*(S) (6). Một heuristic được gọi là admissible khi và chỉ khi h(n) <= h*(n) cho mọi node n.)*

```
# CAU 2d – Cai dat UCS (Uniform Cost Search) va so sanh voi A*
# UCS = A* voi h(n) = 0 cho moi node (tuong duong Dijkstra)
# ==========================================================

def ucs(graph, start, goal):
    """
    Uniform Cost Search.
    Chi dung g(n) - khong dung heuristic.
    Tra ve (duong_di, tong_chi_phi, so_node_expand).
    """
    # TODO: Cai dat UCS bang priority queue, sap xep theo g(n)
    open_list = [(0, start, [start])]
    closed_list = set()
    expand_count = 0
```

```
    while open_list:
        g, current, path = heapq.heappop(open_list)
        if current in closed_list: continue

        expand_count += 1
        closed_list.add(current)

        if current == goal:
            return path, g, expand_count

        for neighbor, cost in graph.get(current, {}).items():
            if neighbor not in closed_list:
                heapq.heappush(open_list, (g + cost, neighbor, path + [neighbor]))
    return None, 0, expand_count

path_u, cost_u, exp_u = ucs(graph, 'S', 'G')
print(f"Kết quả UCS: {' -> '.join(path_u)} | Cost: {cost_u} | Expand: {exp_u}")
pass


# So sanh A* va UCS
path_a, cost_a, exp_a = astar(graph, heuristic, 'S', 'G')
path_u, cost_u, exp_u = ucs(graph, 'S', 'G')

print("=" * 55)
print("SO SANH A* vs UCS")
print("=" * 55)
print(f"{'Thuat toan':<12} {'Duong di':<22} {'Cost':<6} {'#Expand':<10}")
print("-" * 55)
print(f"{'A*':<12} {' -> '.join(path_a):<22} {cost_a:<6} {exp_a:<10}")
print(f"{'UCS':<12} {' -> '.join(path_u):<22} {cost_u:<6} {exp_u:<10}")

# TODO (comment): Nhan xet
# - Hai thuat toan co cung tong chi phi khong? Có, cả hai đều tìm được đường đi tối ưu với tổng chi phí là 8

# - Thuat toan nao expand it node hon? Vi sao? thuật toán A* expand ít node hơ so với UCS. Do A* sử dụng thông
#tin heuristic để ước lượng chi phí còn lại đến đích, giúp nó tập trung tìm kiếm vào những khu vực có triển vọng hơn.
# - Khi nao A* va UCS cho ket qua giong nhau? A* và UCS sẽ cho kết quả giống nhau (cùng đường đi và chi phí tối ưu) khi heu
#của A* là admissible. Ngoài ra, chúng cũng sẽ cho kết quả giống nhau về số lượng node được expand nếu heuristic h(n) của A
#cho tất cả các node (khi đó A* trở thành UCS).
```

```
Kết quả UCS: S -> A -> D -> G | Cost: 8 | Expand: 6
Node  | g   | h   | f
-------------------------
S     | 0   | 7   | 7
A     | 2   | 4   | 6
D     | 3   | 4   | 7
C     | 6   | 2   | 8
G     | 8   | 0   | 8
=======================================================
SO SANH A* vs UCS
=======================================================
Thuat toan   Duong di               Cost   #Expand
-------------------------------------------------------
A*           S -> A -> D -> G       8      5
UCS          S -> A -> D -> G       8      6
```

```
# CAU 2e – What-if: tang cost(A,D) tu 1 len 6
# ==========================================================
import copy
graph_v2 = copy.deepcopy(graph)

# TODO: Cap nhat trong so canh A-D thanh 6 (ca hai chieu vi do thi vo huong)
graph_v2 = copy.deepcopy(graph)
graph_v2['A']['D'] = 6

print("--- Chạy lại A* với cost(A,D)=6 ---")
path_v2, cost_v2, exp_v2 = astar(graph_v2, heuristic, 'S', 'G')




print("=" * 55)
print("A* sau khi tang cost(A,D) tu 1 -> 6")
print("=" * 55)
path_v2, cost_v2, _ = astar(graph_v2, heuristic, 'S', 'G')
print(f"Duong di moi: {' -> '.join(path_v2)}")
print(f"Tong chi phi: {cost_v2}")

# TODO (comment): Duong di co thay doi khong? Vi sao?
#Đường đi tối ưu CÓ thay đổi.
#Đường đi mới: S -> A -> C -> G.
#Tổng chi phí mới: 9.
```

```
#Lý do: Sau khi tăng chi phí tại nhánh D, đường đi qua
#nhánh C trở thành con đường có tổng chi phí thấp nhất (9 < 12 < 13).
```

```
--- Chạy lại A* với cost(A,D)=6 ---
Node  | g   | h   | f
-------------------------
S     | 0   | 7   | 7
A     | 2   | 4   | 6
C     | 6   | 2   | 8
G     | 9   | 0   | 9
=====================================================
A* sau khi tang cost(A,D) tu 1 -> 6
=====================================================
Node  | g   | h   | f
-------------------------
S     | 0   | 7   | 7
A     | 2   | 4   | 6
C     | 6   | 2   | 8
G     | 9   | 0   | 9
Duong di moi: S -> A -> C -> G
Tong chi phi: 9
```

CÂU 3 (4 ĐIỂM) – Minimax cho TicTacToe + Vòng Lặp Game

Phần A – Cài đặt Minimax (2.0đ)

a) (1.5đ) Quan sát code ở ảnh dưới. **Gõ lại toàn bộ và điền 7 chỗ trống** `___(n)___`.

b) (0.5đ) – Tính tay state #1: Cho trạng thái (lượt **X** đi):

```
X | X | _
O | O | _
_ | _ | _
```

Không chạy code, **tính tay** xem Minimax chọn ô nào cho lượt X. *(Gợi ý: X có nước **thắng ngay**, nhưng nếu KHÔNG chọn nó thì O sẽ thắng ở lượt sau!)*

Phần B – Áp dụng (2.0đ)

c) (0.5đ) – Tính tay state #2: Cho trạng thái (lượt **O** đi):

```
X | X | _
_ | O | _
_ | _ | _
```

Không chạy code, tính tay nước đi của O. (Gợi ý: O phải **chặn** X khỏi thắng. X đang đe dọa thắng ở ô nào?)

d) (1.0đ) – Vòng lặp game: Cài đặt hàm `play_game()` cho phép **người chơi (X) đấu với AI (O)**:

- In bảng đầu game.
- Người nhập tọa độ theo cú pháp `"hàng cột"` (ví dụ `"1 2"`).
- AI dùng `minimax()` chọn nước đi tối ưu.
- In bảng sau mỗi lượt.
- Kiểm tra input không hợp lệ (ngoài bàn / ô đã đánh) → yêu cầu nhập lại.
- Khi `terminal(board)` → in kết quả: `X thắng / O thắng / Hòa`.

e) (0.5đ) Tại sao Minimax chậm với bàn lớn hơn 3×3? Đề xuất **một cải tiến** cụ thể (mô tả ngắn, không cần code).

```
from IPython.display import Image, display
display(Image('cau3_code_lan3.png', width=1000))
```

CAU 3 - Dien vao 7 cho trong ___(n) (khong them / xoa dong)

```python
import copy, math
X, O, EMPTY = 'X', 'O', None

def initial_state():
    return [[EMPTY]*3 for _ in range(3)]

def player(board):
    x_count = sum(row.count(X) for row in board)
    o_count = sum(row.count(O) for row in board)
    return X if x_count == o_count else ___(1)___    # TRONG 1

def actions(board):
    return {(i,j) for i in range(3) for j in range(3)
            if board[i][j] == ___(2)___}              # TRONG 2

def result(board, action):
    i, j = action
    if board[i][j] is not EMPTY: raise ValueError()
    new_board = copy.deepcopy(board)
    new_board[i][j] = ___(3)___(board)                # TRONG 3
    return new_board

def winner(board):
    for p in [X, O]:
        for row in board:
            if all(c == p for c in row): return p
        for col in range(3):
            if all(board[r][col] == p for r in range(3)): return p
        if all(board[i][i] == p for i in range(3)): return p
        if all(board[i][___(4)___] == p for i in range(3)): return p  # TRONG 4
    return None

def terminal(board):
    return winner(board) is not None or not actions(board)

def utility(board):
    w = winner(board)
    return 1 if w == X else (-1 if w == O else 0)

def max_value(state):
    if terminal(state): return utility(state)
    v = -math.inf
    for a in actions(state):
        v = max(v, ___(5)___)                         # TRONG 5
    return v

def min_value(state):
    if terminal(state): return utility(state)
    v = ___(6)___                                     # TRONG 6
    for a in actions(state):
        v = min(v, max_value(result(state, a)))
    return v

def minimax(board):
    if terminal(board): return None
    if player(board) == X:
        return max(actions(board),
                   key=lambda a: min_value(result(board, a)))
    else:
        return min(actions(board),
                   key=lambda a: ___(7)___(result(board, a)))  # TRONG 7
```

```python
# CAU 3a – Nhin anh phia tren, go lai toan bo code va dien 7 cho trong
# ============================================================
import copy, math
```

```
import copy, math

X, O, EMPTY = "X", "O", None

# TODO: Go lai va dien 7 cho trong
def initial_state():
    return [[EMPTY]*3 for _ in range(3)]

def player(board):
    x_count = sum(row.count(X) for row in board)
    o_count = sum(row.count(O) for row in board)
    return X if x_count == o_count else O

def actions(board):
    # Sinh ra tap hop cac toa do (i, j) con trong
    return {(i, j) for i in range(3) for j in range(3) if board[i][j] == EMPTY}

def result(board, action):
    i, j = action
    if board[i][j] != EMPTY:
        raise ValueError("O da co nguoi danh")
    new_board = copy.deepcopy(board)
    new_board[i][j] = player(board)
    return new_board

def winner(board):
    # Kiem tra hang
    for row in board:
        if all(c == X for c in row): return X
        if all(c == O for c in row): return O
    # Kiem tra cot
    for j in range(3):
        if all(board[i][j] == X for i in range(3)): return X
        if all(board[i][j] == O for i in range(3)): return O
    # Kiem tra duong cheo
    if all(board[i][i] == X for i in range(3)): return X
    if all(board[i][i] == O for i in range(3)): return O
    if all(board[i][2-i] == X for i in range(3)): return X
    if all(board[i][2-i] == O for i in range(3)): return O
    return None

def terminal(board):
    return winner(board) is not None or not any(EMPTY in row for row in board)

def utility(board):
    win = winner(board)
    if win == X: return 1
    if win == O: return -1
    return 0

def minimax(board):
    if terminal(board): return None
    curr = player(board)
    if curr == X:
        v, best = -math.inf, None
        for act in actions(board):
            score = min_value(result(board, act)) # (1)
            if score > v:
                v, best = score, act
        return best
    else:
        v, best = math.inf, None
        for act in actions(board):
            score = max_value(result(board, act)) # (2)
            if score < v: # (7)
                v, best = score, act
        return best

def max_value(board):
    if terminal(board): return utility(board)
    v = -math.inf
    for act in actions(board):
        v = max(v, min_value(result(board, act))) # (5)
    return v

def min_value(board):
    if terminal(board): return utility(board)
    v = math.inf
    for act in actions(board):
        v = min(v, max_value(result(board, act))) # (6)
    return v
```

```
# Helper de in bang
def print_board(board):
    print("  0 1 2")
    for i, row in enumerate(board):
        print(f"{i} " + " ".join(c if c else '.' for c in row))
    print()

# Kiem tra cau 3b - state #1
state_1 = [
    [X, X, EMPTY],
    [O, O, EMPTY],
    [EMPTY, EMPTY, EMPTY]
]
print("STATE #1 (luot X):")
print_board(state_1)
print(f"Minimax chon: {minimax(state_1)}   (mong doi: (0,2) - X thang ngay hang 0)")
print()

# Kiem tra cau 3c - state #2
state_2 = [
    [X, X, EMPTY],
    [EMPTY, O, EMPTY],
    [EMPTY, EMPTY, EMPTY]
]
print("STATE #2 (luot O):")
print_board(state_2)
print(f"Minimax chon: {minimax(state_2)}   (mong doi: (0,2) - chan X thang hang 0)")
```

```
STATE #1 (luot X):
  0 1 2
0 X X .
1 O O .
2 . . .

Minimax chon: (0, 2)   (mong doi: (0,2) - X thang ngay hang 0)

STATE #2 (luot O):
  0 1 2
0 X X .
1 . O .
2 . . .

Minimax chon: (0, 2)   (mong doi: (0,2) - chan X thang hang 0)
```

Câu 3b – Tính tay state #1 (lượt X):

Bảng:

```
X | X | _
O | O | _
_ | _ | _
```

Ô trống: `(0,2), (1,2), (2,0), (2,1), (2,2)`

Phân tích từng nước đi của X:

- **X đi** `(0,2)` → hàng 0 = `X,X,X` → **X thắng ngay** → utility = +1
- **X đi** `(1,2)` → chặn O (vì O đe dọa hàng 1). Game tiếp tục, kết quả = 0
- **X đi** `(2,0)`, `(2,1)`, **hoặc** `(2,2)` → KHÔNG chặn O → O sẽ đi `(1,2)` → hàng 1 = `O,O,O` → **O thắng** → utility = -1

X là MAX → chọn nước có `min_value` LỚN nhất.
Kết luận: Minimax chọn ô **(0, 2)**

Câu 3c – Tính tay state #2 (lượt O):

Bảng:

```
X | X | _
_ | O | _
_ | _ | _
```

X đang đe dọa thắng tại ô **(0, 2)** (X có 2 quân liên tiếp ở hàng 0).

Nếu O **không chặn**, lượt sau X sẽ thắng → utility = +1 → tệ cho O.
Vậy O (MIN) phải chọn ô để **chặn** X.

Kết luận: Minimax (O) chọn ô **(0, 2)**

```
# CAU 3d – Cai dat vong lap game (nguoi vs AI)
```

```
# ============================================================

def play_game():
    """
    Vong lap game: Nguoi (X) vs AI (O).
    - Nguoi nhap toa do dang 'hang cot' (vd: '1 2')
    - AI dung minimax() chon nuoc di toi uu
    - In bang sau moi luot, kiem tra ket thuc
    """
    board = initial_state()
    print("=" * 40)
    print("TICTACTOE – Nguoi (X) vs AI (O)")
    print("Nhap toa do dang: hang cot (vd: 1 2)")
    print("=" * 40)
    print_board(board)

    while not terminal(board):
        current = player(board)
        if current == X:
            # TODO: Nhan input nguoi choi, parse 'hang cot', validate (trong bang & o trong)
            #       Neu sai -> yeu cau nhap lai
            #       Neu dung -> board = result(board, action)

            while True:
                try:
                    move_str = input(f"Luot {player(board)}. Nhap nuoc di (hang cot): ")
                    row, col = map(int, move_str.split())
                    action = (row, col)

                    if not (0 <= row < 3 and 0 <= col < 3):
                        print("Nuoc di khong hop le: Toa do ngoai ban co. Thu lai.")
                    elif action not in actions(board): # Check if the spot is already taken
                        print("Nuoc di khong hop le: O nay da duoc danh. Thu lai.")
                    else:
                        board = result(board, action)
                        break
                except ValueError:
                    print("Input khong hop le. Hay nhap 'hang cot', vi du '1 2'.")
                except Exception as e:
                    print(f"Loi: {e}. Thu lai.")
         # TODO: AI dung minimax()
         #       In thong bao 'AI di o (i,j)'
         #       board = result(board, action)
        else: # AI player (O)
            print(f"Luot {player(board)}. AI dang tinh toan...")
            action = minimax(board)
            if action:
                board = result(board, action)
                print(f"AI di o: {action[0]} {action[1]}")
            else:
                # Should not happen if terminal is checked before minimax
                print("AI khong the tim thay nuoc di hop le.")
                break

            pass
        print_board(board)

    # TODO: In ket qua: 'X thang!' / 'O thang!' / 'Hoa!'
    final_winner = winner(board)
    if final_winner == X:
        print("X thang!")
    elif final_winner == O:
        print("O thang!")
    else:
        print("Hoa!")
    print("Ket thuc game!")
    pass


# Bo comment dong duoi de chay (tat khi nop bai)
# play_game()
```

Câu 3e – Tại sao Minimax chậm và đề xuất cải tiến:

(Tại sao Minimax chậm: Do thuật toán phải duyệt toàn bộ cây trò chơi. Khi kích thước bàn cờ tăng, số lượng trạng thái (nước đi) bùng nổ theo cấp số nhân, khiến máy tính không thể xử lý hết. Cải tiến cụ thể: Sử dụng kỹ thuật Cắt tỉa Alpha-Beta (Alpha-Beta Pruning). Kỹ thuật này giúp bỏ qua các nhánh không cần thiết mà vẫn đảm bảo tìm được nước đi tối ưu, từ đó tăng tốc độ tính toán đáng kể.)

```
# ============================================================
# CELL NOP BAI – Chay sau khi hoan thanh toan bo de
# Sinh vien KHONG sua cell nay
```

```
# ============================================================
import os, subprocess

try:
    from google.colab import drive
    IS_COLAB = True
except ImportError:
    IS_COLAB = False

NOTEBOOK_NAME = "De_Thi_Lan3.ipynb"

print("Buoc 1: Dang chuyen doi sang PDF...")

result = subprocess.run(
    ["jupyter", "nbconvert", "--to", "webpdf",
     "--allow-chromium-download", "--no-input",
     NOTEBOOK_NAME],
    capture_output=True, text=True
)

pdf_file = NOTEBOOK_NAME.replace(".ipynb", ".pdf")

if result.returncode != 0 or not os.path.exists(pdf_file):
    print("  webpdf that bai, chuyen sang HTML...")
    subprocess.run(
        ["jupyter", "nbconvert", "--to", "html", "--no-input", NOTEBOOK_NAME],
        capture_output=True
    )
    html_file = NOTEBOOK_NAME.replace(".ipynb", ".html")
    if os.path.exists(html_file):
        if IS_COLAB:
            from google.colab import files
            files.download(html_file)
        print(f"\n[HTML] File '{html_file}' da tai xuong.")
        print("=> Mo file HTML trong trinh duyet -> Ctrl+P -> Save as PDF")
    else:
        print("Loi: Khong the chuyen doi. Hay dung File > Print trong Colab.")
else:
    print(f"  Thanh cong! File: {pdf_file}")
    if IS_COLAB:
        from google.colab import files
        files.download(pdf_file)
        print(f"\n[PDF] '{pdf_file}' da tai xuong may tinh.")
    else:
        print(f"\n[PDF] Da luu tai: {os.path.abspath(pdf_file)}")

print("\nDat ten lai file theo dinh dang: MSSV_HoTen.pdf")
```